LAB 11: SOFTWARE METRICS & COMPLEXITY ANALYSIS

Submitted by: Sparsh Tyagi (202401100200308)
Vaishnavi Rajawat, Shubham Patel, Varnit Tomar
Faculty Mentor: Mr. Rishabh Rathore Sir
Academic Year: 2026-27
Department of Computer Science and Engineering | KIET Deemed to be University, Ghaziabad

AIM

To perform quantitative software measurement and complexity analysis for core algorithms of the Multi-Vendor E-Commerce Platform. This includes calculating McCabe's Cyclomatic Complexity $V(G)$, constructing Control Flow Graphs (CFGs), computing Halstead's Software Science Metrics, conducting Function Point (FP) analysis, and evaluating Chidamber & Kemerer (CK) Object-Oriented metrics.

THEORY & MATHEMATICAL FORMULATIONS

Software metrics provide quantitative indicators of software size, complexity, testability, and maintainability.

- **McCabe's Cyclomatic Complexity $V(G)$:** Measures the number of linearly independent paths through a program's control flow graph.

  $$V(G) = E - N + 2P \quad \text{or} \quad V(G) = D + 1 \quad \text{or} \quad V(G) = R$$

  where $E = \text{number of edges}$, $N = \text{number of nodes}$, $P = \text{connected components (typically 1)}$, $D = \text{decision nodes}$, and $R = \text{enclosed regions}$.
- **Halstead Metrics:** Based on primitive counts of unique operators (η_1), unique operands (η_2), total operators (N_1), and total operands (N_2).
  - Program Vocabulary: $\eta = \eta_1 + \eta_2$
  - Program Length: $N = N_1 + N_2$
  - Calculated Length: $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$
  - Volume: $V = N \times \log_2 \eta$ (bits)
  - Difficulty: $D = (\eta_1 / 2) \times (N_2 / \eta_2)$
  - Effort: $E = D \times V$ (elementary mental discrimination steps)
  - Estimated Time to Code: $T = E / 18$ (seconds)

ANALYZED SOURCE CODE: MULTI-VENDOR ORDER SPLITTING ENGINE

Below is the baseline TypeScript function evaluated for Cyclomatic Complexity and Halstead Metrics:

```
     function processMultiVendorOrder(cart: CartItem[], coupon: Coupon | null): VendorSubOrderResult { if
(!cart || cart.length === 0) { throw new Error("Cart is empty"); } let grossTotal = 0; const vendorGroups =
new Map(); for (const item of cart) { if (item.stockQty < item.requestedQty) { return { success: false,
reason: "OUT_OF_STOCK" }; } grossTotal += item.price * item.requestedQty; if
(!vendorGroups.has(item.vendorId)) { vendorGroups.set(item.vendorId, []); }
vendorGroups.get(item.vendorId)!.push(item); } let discount = 0; if (coupon && coupon.isValid && grossTotal
>= coupon.minSpend) { discount = (grossTotal * coupon.percentage) / 100; } return { success: true, groups:
vendorGroups, finalTotal: grossTotal - discount }; }
```

CONTROL FLOW GRAPH (CFG) & CYCLOMATIC COMPLEXITY ANALYSIS

Nodes in the CFG represent sequential basic statement blocks, and directed edges represent control transfers.

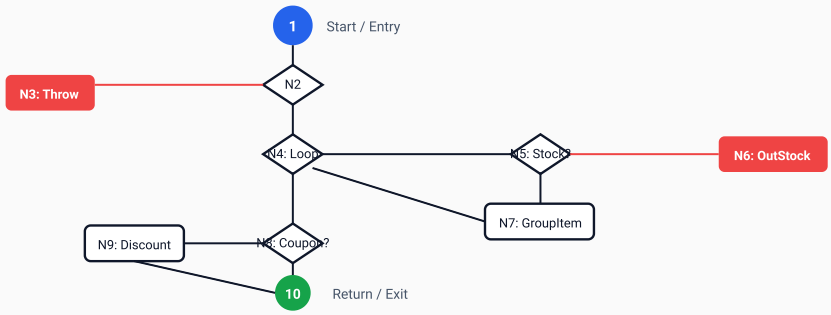


FIGURE 11.1: CONTROL FLOW GRAPH (CFG) FOR MULTI-VENDOR ORDER SPLITTING ENGINE

CYCLOMATIC COMPLEXITY CALCULATION RESULTS

Calculation Method	Formula Applied	Result	Risk Level
Edge-Node Method	$V(G) = E - N + 2P = 13 - 10 + 2(1)$	**$V(G) = 5$**	Low Risk / Structured
Predicate Node Method	$V(G) = \text{Predicates } (D) + 1 = 4 + 1$	**$V(G) = 5$**	Highly Testable
Region Count Method	$V(G) = \text{Enclosed Regions } (R) + \text{Outer Region}$	**$V(G) = 5$**	Clean Logic

HALSTEAD SOFTWARE SCIENCE METRICS CALCULATION

Halstead Primitive Parameter	Identified Tokens	Count	Derived Metric & Computed Value
Unique Operators (η_1)	`if`, `for`, `throw`, =, +, *, !, &&, <, >=	$\eta_1 = 10$	**Vocabulary (η):** $\eta_1 + \eta_2 = 25$
Unique Operands (η_2)	`cart`, `coupon`, `grossTotal`, `vendorGroups`, `item`, `price`, etc.	$\eta_2 = 15$	**Length (N):** $N_1 + N_2 = 56$
Total Operator Occurrences (N_1)	Sum of operator occurrences in code body	$N_1 = 26$	**Volume (V):** $56 \times \log_2(25) = \mathbf{260.05 \text{ bits}}$
Total Operand Occurrences (N_2)	Sum of variable/literal occurrences	$N_2 = 30$	**Difficulty (D):** $(10/2) \times (30/15) = \mathbf{10.0}$
Effort (E) & Time (T)	$E = D \times V$; $T = E / 18$ seconds	-	**Effort (E):** $\mathbf{2600.5 \text{ discriminations}}$ **Time (T):** $\mathbf{144.47 \text{ seconds}}$

CHIDAMBER & KEMERER (CK) OBJECT-ORIENTED METRICS

Metric	Description	Target Class Evaluation	Design Assessment
WMC	Weighted Methods per Class	`OrderSplitterService`: 4 methods (avg $V(G) = 3$)	**WMC = 12** (Low complexity)
DIT	Depth of Inheritance Tree	`Customer` $\rightarrow$ `User` $\rightarrow$ `Object`	**DIT = 2** (Optimal reuse)
NOC	Number of Children	`User` base class has 3 children (Customer, Vendor, Admin)	**NOC = 3** (Balanced hierarchy)
CBO	Coupling Between Object Classes	`CheckoutController` couples to 4 service classes	**CBO = 4** (Low coupling)
LACK	Lack of Cohesion in Methods	Disjoint method pairs operating on independent fields	**LACK = 0** (High cohesion)

CONCLUSION

The software metrics analysis confirms that core algorithms of the Multi-Vendor E-Commerce Platform are well-structured, maintainable, and highly testable. The Cyclomatic Complexity of $V(G) = 5$ confirms a low-risk profile requiring exactly 5 test cases for 100% path coverage. Halstead and CK metrics demonstrate low coupling, high cohesion, and minimal maintenance effort.